

게이트가 우리 커널을 막아 세웠습니다

C-3.1 게이트가 첫 커널 커밋을 멈춰 세웠고, 그 이유가 정확했습니다. `tenant_scoped`는 `request`를 못 찾으면 그냥 통과시키므로, 커널에 붙이면 표식만 남고 아무것도 안 막습니다 — 착시 ①(부착물)의 재현입니다. 게이트가 자기가 만들 뻔한 착시를 잡았습니다.

1 검출은 없던 게 아니라 버려지고 있었다

AI 서버는 `Detection{x,y,w,h,label,confidence,color}`를 이미 돌려주는데 `grpc_client.py:147`이 이미지만 디코딩하고 나머지를 통째로 버린다 — 추가 AI 개발 없이 배선만으로 탐지가 살아난다.

2 표가 없는데 시험은 초록이었다 — 다섯 번째 착시

`DetectionEvent` 표가 DB에 없다(모델 14 / 표 0). `--nomigrations`가 모델 선언에서 테스트 DB를 만들어 준 덕이었다. 재는 자리가 실물이 아니었다.

3 독스트링은 "저장한다", 본문은 비었고 True를 낸다

`detect_and_save`. 부르는 쪽이 "저장됐다"고 믿고 다음을 싹는다 — 조용한 성공이 가장 나쁘다.

판정 5건

D-281 P-K1-1 — C-3.1을 계층별로 나눈다

승인부 · 문언 개정

커널 계층에서 request 기반 데코레이터 금지. 대신 테넌트 없이는 호출 자체가 불가능한 시그니처로 만든다. 데코레이터는 "불었는가"만 보지만, 필수 인자는 부르는 쪽이 테넌트를 알아야만 호출된다 — 표식이 아니라 구조이고, 우회할 자리가 없다.

```
# HTTP 계층(뷰) — request 가 실제로 있으므로 데코레이터 유효
@tenant_scoped()
def event_detail(request, pk): ...

# 커널 계층(L3) — 스코프를 키워드 전용 필수 인자로
def list_events(*, scope: TenantScope, ...): # scope 없으면 호출 자체가 불가
    ...
x list_events()          → TypeError (부르는 순간 실패)
✓ list_events(scope=s)   → 테넌트를 아는 호출만 통과
```

네 제안(통과 형태로 문지기 호출을 더해 요구 수준을 올림)도 함께 채택 — 시그니처가 1차, 문지기가 2차. 게이트에 "커널 공개 함수 중 scope 인자 없는 것 = fail"을 추가하라. C-3.1 문언은 이 판정으로 개정된 것으로 본다(임의 이행 아님).

D-283 0016_detectionevent 적용 승인 — W2-2보다 먼저

조건 4건

신규 표 생성은 기존 데이터를 건드리지 않으므로 백필과 성격이 다르다. "표 없이 배선하면 첫 검출에서 죽는다"는 판단이 옳다. 조건: ① sqlmigrate로 실행될 DDL 먼저 출력·첨부 ② 역방향 가능 여부 확인(불가면 사유 적고 멈춤) ③ 로컬 복제본만 스냅샷 1회 ④ 적용 후 D-282 게이트 green 확인 — 재발 방지가 곧 검증이다.

D-284 호출처 2곳 시정 + 거짓 성공 반환 금지

원칙 신설

(1) grpc_client.py:147 — detections를 K1 이벤트로 넘긴다. proto 실측이 이미 근거다. W2-2 배선의 핵심이자 K1 호출처 ①의 실질.
(2) detect_and_save — 구현이 없는 함수는 성공을 반환하지 않는다. 빈 본문은 NotImplementedError를 던진다. → 저장소 전수로 "본문 없이 True/None을 반환하는 공개 함수" 목록을 1회 조사하라.

D-285 DA-05 처분 + 트립와이어 설계 정분화

인정

실호출 0/5는 네 잘못이 아니다 — 키가 이 PC에 없다. 추정으로 채워지 않은 것은 D-280 준수. 키가 오면 한 줄로 끝나는 상태까지 세워둔 것도 인정.
★ "수가 아니라 이름으로 잠근다"를 정분화한다 — 수 래치는 라우트 하나가 지워질 때마다 새 라우트가 들어올 자리를 만든다는 지적이 정확하다. 앞으로 모든 커버리지 게이트는 이름 집합 기준. 개수 기준 금지. 무방비 라우트를 심어 양쪽이 멈추는 것까지 확인한 절차(양성 대조)도 정분. 계층 CI가 대상 0건일 때 판정기 자체를 시험한 것(대조 14/14)은 D-277의 자발적 적용 — 잘했다.

재는 자리가 실물이 아니었다

D-282 — DetectionEvent 표가 DB에 없는데(모델 14건 / 표 0건) 시험은 초록이었습니니다. --nomigrations가 모델 선언에서 테스트 DB를 만들어 줬기 때문입니다. 앞의 넷과 다른 종류입니다 — 시험 환경이 실물과 다르면 초록은 실물을 말하지 않습니다.

착시 ①
부착물
466/466 부착=완결
D-249

착시 ②
시나리오 수
5/5인데 대상 9종
D-262

착시 ③
모수와 술어
모수가 2, 술어가 FK 못 볼
D-271

착시 ④
탐지기 자체
실패할 수 없는 시험
D-277

착시 ⑤ · 이번
환경
재는 자리가 실물이 아님
D-282

①~③ = 무엇을 얼마나 잤나 · ④ = 재는 기계가 작동하나 · ⑤ = 재는 자리가 실물인가. 세 층이 다 통과해야 초록이 사실이 됩니다.

D-282 대응 — 마이그레이션 정합 게이트

조치	내용
게이트 신설	python manage.py makemigrations --check --dry-run → 미반영 모델 변경이 있으면 exit 1. "모델 14 / 표 0" 같은 격차가 조용히 유지되는 것을 막는다
--nomigrations 위치 규정	속도용 보조 수단이지 정보 판정 환경이 아니다. 최소 1개 시험 경로는 실제 마이그레이션을 적용한 DB에서 돌아야 한다
skip 1 해소	컨테이너에 scripts/ 미마운트로 계층 게이트 호출 불가 → docker-compose에 마운트 추가. 사실대로 남긴 것은 인정

\$+ 게이트 현황 — 6종 → 8종

게이트	막는 것	상태
tenant_scope	스코프 없는 경로 · + 커널 scope 인자 없음(D-281)	가동중 → 확장
layers	App → 커널 직접 import	가동중 (대조 14/14)
classification	등록부 미참조 쓰기	가동중
timeout	타임아웃 없는 외부 호출	가동중
kernel_map	미지 커널이 검증 밖으로	가동중
manifest	레지스트리 불일치	가동중
migration_sync	모델과 표의 격차(착시 ⑤)	신설 — D-282
secrets(키 형태)	gitleaks 룰 4건 · 양성 4/4 · 전수 오탐 0	세움 — DA-05

실행 순서 · 대표 조치

§5 순서

마이그레이션 → 배선 → 병렬

- 1

D-281~285 정서 — 커밋 1개
- 2

D-281 커널 시그니처 개정 — scope 키워드 전용 필수 인자 + verify_tenant_scope에 커널 검사 추가. 게이트가 막았던 K1 첫 커밋을 이 형태로 통과시켜라
- 3

D-282 makemigrations --check 게이트 신설 + scripts/ 마운트 추가(skip 해소)
- 4

D-283 0016 적용 — DDL 첨부 → 역방향 확인 → 로컬 적용 → 게이트 green
- 5

D-284 W2-2 배선 — grpc_client detections 살리기 + detect_and_save 정직화 + 빈 본문 True 전수 조사
- 6

K1 done 확정 — 이중 AC green + 호출처 2곳이 실제로 이벤트를 넘기는 것 확인 (표식 아님)
- 7

K2 II K6 병렬 착수

이번 순서의 의미 — ⑤에서 detections를 살리는 순간, AI를 새로 만들지 않고도 탐지가 제품에 들어옵니다. 12스프린트 동안 "탐지가 약하다"고 여겨 온 것이 실은 이미 오는 데이터를 버리고 있던 것이었습니다 — K1 커널의 첫 배당금이 여기서 나옵니다.

§6 대표게 — DA-05를 여는 2건

이것만 오면 한 줄로 끝납니다

항목	하실 일
① 환경변수 2개	기상청 키와 공공데이터포털 키를 C:\GuardianX-vault\ 또는 환경변수로 이 PC에 두십시오. 값은 저에게 보여주실 필요 없습니다 — Code가 vault에서 읽어 비밀저장소로 옮깁니다. 변수 이름은 Code가 docs/agent/evidence/DA-05/README.md §2에 적어 뒀습니다
② 서비스명 확인	올려주신 문서에 국립공원 안전정보와 아동센터·돌봄센터 안전정보가 같은 엔드포인트(/getNlprkSafetyInfoSearch_4)로 적혀 있습니다. 둘 중 최소 하나는 다른 API입니다. 포털 마이페이지의 실제 활용신청 내역에서 두 서비스의 정확한 이름을 확인해 주십시오 — 추정하면 2027.2에 엉뚱한 API를 쓰게 됩니다(Code 지적 그대로)

한 문장 — 이번 턴에 게이트가 우리 편의 코드를 막았습니다. 그게 게이트를 세운 이유입니다 — 남의 실수만 잡는 검사는 오래 못 갑니다. 자기 커널을 막을 수 있는 게이트라야 다음 커널도 지킵니다.